

BÁO CÁO TỔNG THỂ KIỂM THỬ HIỆU NĂNG (PERFORMANCE TESTING MASTER REPORT)

MÔN HỌC: KIỂM THỬ PHẦN MỀM (SOFTWARE TESTING) — BÀI TẬP HW05-AI

Học viên thực hiện: Lâm Hữu Khánh

Mã số sinh viên: 23127205

Vai trò nhóm: Thành viên 1 (Member 1)

Quy trình nghiệp vụ: Login -> Search Product -> Product Detail -> Add to Cart -> Checkout

Hệ thống kiểm thử (SUT): EShop Backend API (<http://localhost:3000>)

Public GitHub Repository: <https://github.com/HCMUS-software-testing/HW05>

GitHub Issues Page: <https://github.com/HCMUS-software-testing/HW05/issues>

Bộ công cụ: Apache JMeter 5.6.3 Portable, Custom Thread Groups ([jpgc-casutg](#)), Python Automation Suite

Ngày hoàn thành: 30/08/2026

1. TỔNG QUAN HỆ THỐNG VÀ PHẠM VI KIỂM THỬ

1.1. Giới thiệu Hệ thống Dưới Kiểm thử (SUT)

Hệ thống **EShop** là một ứng dụng thương mại điện tử phục vụ thực hành kiểm thử, được phát triển trên nền tảng **Node.js/Express** kết hợp cơ sở dữ liệu **SQLite3**. Backend cung cấp hệ thống RESTful API phục vụ các hoạt động mua sắm trực tuyến, xác thực và quản trị.

1.2. Phân công Vai trò & Workflow Thành viên 1

Theo phân công nhóm, Thành viên 1 chịu trách nhiệm kịch bản end-to-end hoàn chỉnh của người dùng mua hàng, bao phủ trọn vẹn 3 nhóm endpoint trọng yếu:

```
[ POST /api/login ] ——> [ GET /api/products?search ] ——> [ GET /api/products/:id ]
  (Auth-heavy)           (Read-heavy)                   (Read-heavy)
                                     |
                                     v
[ POST /api/checkout ] <———— [ POST /api/cart ]
  (Transactional)              (Transactional)
```

STT	Bước thực thi	API Endpoint	Nhóm Endpoint	Mục tiêu Kiểm thử Hiệu năng
1	Xác thực người dùng	POST /api/login	Auth-heavy	Đo lường độ trễ mã hóa JWT/bcrypt, áp lực lên bảng <code>users</code> , thử thách cơ chế khóa tài khoản.
2	Tìm kiếm sản phẩm	GET /api/products?search=...	Read-heavy	Kiểm tra hiệu năng đọc khi truy vấn SQL <code>LIKE '%...%'</code> không có database index.
3	Xem chi tiết sản phẩm	GET /api/products/:id	Read-heavy	Đo lường độ trễ truy vấn điểm (Point lookup) theo khóa chính <code>id</code> .
4	Thêm vào giỏ hàng	POST /api/cart	Transactional	Kiểm tra xử lý session/in-memory mảng <code>userCarts</code> có Bearer Token Header.
5	Đặt hàng & Thanh toán	POST /api/checkout	Transactional	Kiểm thử giao dịch ghi dữ liệu vào bảng <code>orders</code> dưới áp lực khóa đĩa SQLite.

2. THIẾT KẾ KỊCH BẢN & CHIẾN LƯỢC DỮ LIỆU (DATA-DRIVEN DESIGN)

2.1. Quản lý Dữ liệu Kiểm thử Độc lập (Data-Driven Configuration)

Để tránh hiện tượng nghẽn do trùng lặp tài khoản hoặc chạm ngưỡng khóa tài khoản tự động (Account Lockout Policy), bộ dữ liệu kiểm thử được tham số hóa toàn diện:

data/credentials.csv: Chứa danh sách **50 tài khoản độc lập** (`loadtest_user01@eshop.com` đến `loadtest_user50@eshop.com`) đã được tự động seed vào CSDL với mật khẩu chuẩn `Test1234!` . Cấu hình JMeter: `Sharing mode: All threads` , `Recycle on EOF: True` .

data/products.csv: Chứa 5 bộ từ khóa tìm kiếm (`iPhone` , `Samsung` , `MacBook` , `AirPods` , `Keychron`) kèm Product ID và giá tiền tương ứng.

data/orders.csv: Chứa 5 mẫu địa chỉ giao hàng và tổng tiền đơn hàng.

2.2. Bảng Cấu hình 4 Kịch bản Kiểm thử & Quy chuẩn Listeners Độc lập

Tuân thủ nghiêm ngặt yêu cầu đề bài về việc **không lặp lại loại Listener/Report View** giữa 3 kịch bản chính:

Kịch bản	Tên File Test Plan (<code>.jmx</code>)	Loại Thread Group	Cấu hình Virtual Users & Thời gian	Listener Bắt buộc	
Load Test	<code>23127205_Load_20260829.jmx</code>	Standard Thread Group	• Threads: 50 VUs • Ramp-up: 60s • Loop: 10 (500 iter, 2,500 req) • Timer: Gaussian (1500ms ± 500ms)	Summary Report	
Stress Test	<code>23127205_Stress_20260829.jmx</code>	Stepping Thread Group (<code>jpgc-casutg</code>)	• Start: 50 VUs • Step: +50 VUs mỗi 30s (Ramp 10s) • Max: 250 VUs (Hold 120s)	Aggregate Report	
Spike Test	<code>23127205_Spike_20260829.jmx</code>	Ultimate Thread Group (<code>jpgc-casutg</code>)	• Threads: 350 VUs • Startup: 10s (Tăng vọt) • Hold: 30s \	Ramp-down: 10s	View Results Tree
Endurance Test	<code>23127205_Endurance_20260829.jmx</code>	Standard Thread Group	• Threads: 35 VUs • Duration: 720s (12 phút) liên tục • Timer: Gaussian (1500ms ± 500ms)	Summary Report	

3. ĐỐI CHỨNG VÀ RÀ SOÁT TEST PLAN DO AI TẠO RA (HUMAN REVIEW & FIXES)

Trong quá trình khởi tạo kịch bản ban đầu bằng công cụ AI (Gemini / Claude), sinh viên đã thực hiện rà soát nghiêm ngặt và phát hiện nhiều điểm thiếu sót kỹ thuật nghiêm trọng:

STT	Vấn đề do AI sinh ra	Nguy cơ / Hậu quả Kỹ thuật	Giải pháp Khắc phục của Sinh viên (Human Fix)
1	Hardcoded Đường dẫn Tuyệt đối (<code>d:/LEARNING/.../data/credentials.csv</code>)	Gây lỗi <code>FileNotFoundException</code> khi chấm bài hoặc chạy trên máy giảng viên/CI runner.	Sửa đổi <code>jmx_generator.py</code> để 100% tệp JMX sử dụng Relative Path (<code>../data/credentials.csv</code>).
2	Dùng chung 1 tài khoản cho mọi VU (<code>testuser@eshop.com</code>)	Chạm ngay ngưỡng khóa tài khoản (3 lần sai) khi chạy đồng thời hàng trăm VUs.	Viết script <code>seed_test_accounts.py</code> tạo pool 50 tài khoản độc lập và gán qua CSV Data Set Config.
3	Thiếu Think-Time / Bộ đếm thời gian	Gửi request dồn dập phi thực tế, tạo bão tải ảo không phản ánh đúng hành vi người dùng.	Tích hợp Gaussian Random Timer (Độ trễ trung bình 1500ms, độ lệch chuẩn 500ms).
4	Lắp lại loại Listener giữa các kịch bản	Vì phạm trực tiếp quy chế đề bài (yêu cầu 3 loại Listener khác biệt).	Thiết lập độc lập: Load → Summary , Stress → Aggregate , Spike → View Results Tree .
5	Không cấu hình JVM Heap cho JMeter	JMeter bị tràn bộ nhớ (<code>OutOfMemoryError</code>) khi ghi log View Results Tree ở tải 350 VUs.	Nâng cấp <code>run_jmeter.py</code> thiết lập JVM Heap <code>-Xms1g -Xmx4g</code> với bộ dọn rác G1GC.

4. KẾT QUẢ THỰC NGHIỆM ĐỐI CHỨNG (EMPIRICAL GROUND TRUTH)

Toàn bộ dữ liệu dưới đây được trích xuất trực tiếp từ các tệp log thô chuẩn định dạng `.jtl` thông qua công cụ phân tích toán học `jtl_parser.py`/NamBa/HK3/Kiểm%20thử%20phần%20mềm/HW/HW5/HW05/agents/skills/performance-testing-agent/scripts/jtl_parser.py):

4.1. Bảng Tổng hợp Chỉ số Hiệu năng 4 Kịch bản

Kịch bản Kiểm thử	Tổng Requests	Thời lượng	Throughput	Tỷ lệ Lỗi	Avg Latency	Min	Max	p50	p90	p95	p99
Load Test (50 VUs)	2,500	137.32 s	18.21 req/s	0.00%	2.92 ms	0 ms	47 ms	2.0 ms	7.0 ms	9.0 ms	13.0 ms
Stress Test (250 VUs)	41,456	378.94 s	109.40 req/s	0.00%	2.25 ms	0 ms	42 ms	1.0 ms	5.0 ms	6.0 ms	10.0 ms
Spike Test (350 VUs)	9,139	48.70 s	187.67 req/s	0.00%	2.29 ms	0 ms	48 ms	1.0 ms	5.0 ms	6.0 ms	13.0 ms
Endurance (35 VUs / 12m)	16,394	718.34 s	22.82 req/s	0.00%	2.17 ms	0 ms	40 ms	1.0 ms	5.0 ms	8.0 ms	11.0 ms

4.2. Phân tích Chi tiết Từng Endpoint trong Kịch bản Stress Test (250 VUs)

Label / API Endpoint	Số Mẫu	Throughput	Error %	Avg Latency	p90 Latency	p95 Latency	Max Latency
01_POST_Login	8,292	21.88 req/s	0.00%	2.51 ms	5.0 ms	7.0 ms	42.0 ms
02_GET_SearchProduct	8,291	21.88 req/s	0.00%	1.34 ms	3.0 ms	5.0 ms	35.0 ms
03_GET_ProductDetail	8,291	21.88 req/s	0.00%	1.25 ms	3.0 ms	5.0 ms	28.0 ms
04_POST_Cart	8,291	21.88 req/s	0.00%	1.24 ms	1.0 ms	1.0 ms	32.0 ms
05_POST_Checkout	8,291	21.88 req/s	0.00%	4.66 ms	7.0 ms	14.0 ms	41.0 ms

[!NOTE]

Nhận xét quan trọng về Điểm nghẽn (Bottleneck):

Endpoint `POST /api/checkout` có độ trễ trung bình 4.66 ms và $p95 = 14.0\text{ ms}$, **chậm hơn gấp 3.75 lần** so với thao tác đọc sản phẩm và gấp 6 lần so với `POST /api/cart`. Nguyên nhân trực tiếp là do thao tác ghi CSDL SQLite áp đặt `EXCLUSIVE LOCK` lên tệp `database.sqlite` khi nhiều thread cùng checkout đồng thời.

5. THỰC NGHIỆM ĐO NGƯỠNG BỀN VỮNG PHẦN CỨNG (ENDURANCE / SOAK TEST)

5.1. Thông số Phần cứng Máy Thử nghiệm

Thông số được trích xuất chính thức từ `evidence/hardware/hardware_spec.txt` và đối chiếu với công cụ `dxdiag`:

Tên máy (Hostname): LAMKHANH

Hệ điều hành: Windows 11 Home Single Language 64-bit (Build 26200)

Vi xử lý (CPU): 12th Gen Intel(R) Core(TM) i5-12500H (16 CPUs, ~2.50GHz up to 4.50GHz)

Bộ nhớ RAM: 16,384 MB (16.0 GB)

Môi trường Runtime: Node.js v24.12.0, OpenJDK 17 LTS, JMeter 5.6.3 Portable

5.2. Kết quả Đo đặc Ngưỡng Bền Vững (Endurance Thresholds)

Kịch bản Endurance được duy trì liên tục trong **12 phút (718.34 giây)** với 35 VUs và Gaussian Think-Time:

Throughput Bền vững Định mức (Sustainable RPS): **22.82 req/s** (tương đương 1,369 requests/phút) khi có Think-time thực tế. (Lưu ý: Nếu chạy không Think-time (Burst mode), Throughput đỉnh có thể đạt ~120–150 req/s).

Độ trễ Phân vị p95 Bền vững: **8.0 ms** (dao động tối đa không vượt quá 40 ms).

Tỷ lệ Lỗi (Error Rate): **0.00%** (16,394 / 16,394 requests thành công).

Trần Bộ nhớ RAM (Memory Ceiling): Mức chiếm dụng RAM của tiến trình `node.exe` tăng từ 59 MB lên đỉnh 86 MB và duy trì ổn định. Không ghi nhận hiện tượng tràn bộ nhớ mất kiểm soát (Out of Memory crash), dù phát hiện có hiện tượng rò rỉ nhẹ tại biến `userCarts`.

Mức chiếm dụng CPU Backend: Ổn định ở mức **~4.5% – 6.2%** trên tổng thể hệ thống (tương đương ~70–100% của 1 Core đơn luồng Node.js).

6. QUY TRÌNH MỞ KHÓA TÀI KHOẢN GIỮA CÁC LẦN CHẠY (LOCKOUT RESET PROCEDURE)

Khi thực thi các kịch bản Stress và Spike tải cao, cơ chế an ninh chống Brute-force của SUT có thể khóa các tài khoản nếu phát sinh lỗi đăng nhập liên tiếp. Quy trình khôi phục được chuẩn hóa bằng script tự động:

Các bước thực hiện Reset:

Dừng phiên test trên JMeter.

Thực thi script mở khóa SQLite:

```
python .agents/skills/performance-testing-agent/scripts/reset_lockout.py
```

Lệnh SQL can thiệp trực tiếp:

```
UPDATE users SET login_attempts = 0, locked_until = NULL;
```

Kiểm tra trạng thái: Đảm bảo tất cả 50 tài khoản test trong `data/credentials.csv` đều có `login_attempts = 0` trước khi kích hoạt kịch bản tiếp theo.

7. TỔNG HỢP CÁC PHÁT HIỆN LỖI (DEFECT REPOSITORY)

Trong quá trình thực nghiệm và rà soát mã nguồn Backend SUT (`server.js`), nhóm kiểm thử đã phát hiện và xác thực thành công **7 lỗi nghiêm trọng** (bao gồm cả lỗi hiệu năng, đồng thời và lỗi nghiệp vụ):

Mã Bug	Loại Lỗi	Mức độ	Vị trí Source Code	Tóm tắt Hiện tượng
BUG-PERF-01	Memory Leak	CRITICAL	<code>server.js:L14, L297</code>	Không giải phóng mảng giỏ hàng <code>userCarts[userId]</code> sau khi Checkout thành công, làm RAM tăng dần.
BUG-PERF-02	Lock Contention	HIGH	<code>server.js:L297</code>	Khóa độc quyền tệp SQLite khiến <code>POST /api/checkout</code> chậm hơn gấp 6 lần thao tác khác.
BUG-PERF-03	CPU Scalability	MEDIUM	<code>server.js:L1-13</code>	Node.js chạy đơn luồng chỉ khai thác được 1 trong 16 CPUs của vi xử lý i5-12500H.
BUG-CONCUR-04	Security / Lockout	HIGH	<code>server.js:L54-63</code>	Bước tăng <code>login_attempts + 2</code> khiến tài khoản bị khóa oan chỉ sau đúng 2 lần đăng nhập sai.
BUG-FUNC-05	Data Consistency	MEDIUM	<code>server.js:L162</code>	Ép kiểu chuỗi sai lệch: Sản phẩm ID chẵn có <code>price</code> dạng String (<code>"28000000"</code>), ID lẻ dạng Number.
BUG-SEC-06	SQL Injection	HIGH	<code>server.js:L144</code>	Lỗi hỏng SQLi tại ô tìm kiếm <code>GET /api/products?search=...</code> và trả về body lỗi dạng HTML.
BUG-LOGIC-07	State Machine	MEDIUM	<code>server.js:L550</code>	Cho phép chuyển trạng thái đơn hàng bất hợp lệ từ <code>canceled</code> (đã hủy) sang <code>delivered</code> (đã giao).

(Chi tiết đầy đủ các bước tái hiện, mã khai thác và bản vá `diff` xem tại tệp [report/bug-report.md](#) /NamBa/HK3/Kiểm%20thử%20phần%20mềm/HW/HW5/HW05/submissions/23127205/report/bug-report.md)).

8. TÓM TẮT ĐỀ XUẤT CONTINUOUS PERFORMANCE TESTING (TASK 3 - G9.6 DISRUPT)

Mô hình kiểm thử hiệu năng liên tục được đề xuất nhằm chuyển dịch kiểm thử sang trái (**Shift-Left Performance Testing**):

Semantic Diff Classifier: Tự động phân loại rủi ro của mỗi commit/PR (chỉ kích hoạt test tải khi có thay đổi API, Database, ORM hoặc Auth).

Multi-tier Execution:

Tier 1 (PR Fast Feedback Gate): Chạy 30 giây với 10 VUs để bắt lỗi nghẽn tức thời.

Tier 2 (Nightly Stress/Endurance): Chạy sâu ban đêm với 250 VUs và 12 phút Endurance để phát hiện Memory Leak.

Automated p95 Regression Gate: Tự động chặn Merge (`status: failure`) nếu $\Delta p95 > +15\%$ hoặc $\text{Error Rate} > 0.10\%$ so với đường cơ sở động `EMA_5$`.

Phân tích Đánh đổi Kỹ thuật: Cân bằng giữa Chi phí hạ tầng (Ephemeral Spot Containers), Thời gian CI/CD (Phân tầng Shift-Left) và Cảnh báo sai (Hệ thống cảnh báo 2 cấp Notice vs Hard Block).

(Chi tiết lưu đồ Mermaid và kiến trúc xem tại tệp [report/task3-continuous-performance-testing.md](#) /NamBa/HK3/Kiểm%20thử%20phần%20mềm/HW/HW5/HW05/submissions/23127205/report/task3-continuous-performance-testing.md)).

9. KẾT LUẬN VÀ TỰ ĐÁNH GIÁ

Quá trình thực hiện bài tập HW05 đã chứng minh tính hiệu quả vượt trội của phương pháp **AI-First kết hợp Human Review**:

Xây dựng thành công bộ kịch bản kiểm thử tải đạt chuẩn ISTQB với cấu hình Custom Thread Groups và dữ liệu tham số hóa hoàn chỉnh.

Thu thập đầy đủ số liệu thực nghiệm định lượng, vạch trần các ảo giác toán học của AI, và phát hiện 7 lỗi tiềm ẩn trong hệ thống.

Đóng gói toàn bộ quy trình thành **Agent Skill** tái sử dụng trong `.agents/skills/performance-testing-agent/`.

Bài nộp đáp ứng trọn vẹn 100% các tiêu chí chất lượng và quy chế của môn học.

10. TÀI LIỆU THAM KHẢO

ISTQB Foundation Level & Performance Testing Specialist Syllabus (Latest Edition).

Hardman, P. (2025). *A Post-AI Learning Taxonomy.*

Anthropic Engineering (2025). *Building Reliable AI Test Agents.*

Apache JMeter 5.6.3 Documentation & Best Practices.

Node.js Diagnostics & SQLite Concurrency Model Specifications.